Failure (a bad split, and a k comparison) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `chunking_practice.py` (Failure section): it's tempting to treat "just raise `k`" (retrieve more chunks) as a fix once it appears to work on a tiny test document, because with only a handful of chunks total, a bigger `k` can accidentally pull in everything relevant. This exercise builds a larger, realistic corpus and makes you watch that same fix fail — raising `k` stops helping once the right chunk is buried among many irrelevant ones — before you carry the assumption into the Build Task. **If not:** the Build Task's knowledge base would inherit a `k`-bump habit that quietly stops working the moment the corpus grows past a few dozen chunks.

All examples below use:

```
# chunking_practice.py — Failure section
TEST_DOCUMENT = (
    "Our team ships a small internal newsletter every month. "
    "The Treaty of Lisbon was signed on 13 December 2007 in Lisbon, Portugal, "
    "marking a major change to how the European Union is governed. "
    "Most readers skip the history section and go straight to the recipe "
    "at the bottom."
)
QUESTION = "When and where was the Treaty of Lisbon signed?"
```

Solution

```
# chunking_practice.py — Failure section
import math
import time
from openai import OpenAI

client = OpenAI()

def chunk_by_chars(text: str, chunk_size: int) -> list[str]:
    chunks: list[str] = []
    start = 0
    while start < len(text):
```

```
        chunks.append(text[start:start + chunk_size])
        start += chunk_size
    return chunks

def get_embedding(text: str) -> list[float]:
    response = client.embeddings.create(
        model="text-embedding-3-small", input=text
    )
    return response.data[0].embedding

def cosine_similarity(a: list[float], b: list[float]) -> float:
    dot = 0.0
    for x, y in zip(a, b):
        dot = dot + x * y
    squares = 0.0
    for x in a:
        squares = squares + x * x
    norm_a = math.sqrt(squares)
    squares = 0.0
    for x in b:
        squares = squares + x * x
    norm_b = math.sqrt(squares)
    return dot / (norm_a * norm_b)

def retrieve(
    question: str, chunks_with_embeddings: list[tuple[str, list[float]]], k: int
) -> list[tuple[float, str]]:
    question_embedding = get_embedding(question)
    scored: list[tuple[float, str]] = []
    for chunk_text, embedding in chunks_with_embeddings:
        score = cosine_similarity(question_embedding, embedding)
        scored.append((score, chunk_text))
    scored.sort(reverse=True)
    return scored[:k]

def build_index(text: str, chunk_size: int) -> list[tuple[str, list[float]]]:
    chunks = chunk_by_chars(text, chunk_size)
    results = []
    for chunk in chunks:
        results.append((chunk, get_embedding(chunk)))
    return results

# Build the index once (chunk + embed), then look at the top result.
chunks_with_embeddings = build_index(TEST_DOCUMENT, 40)

print("k=1 result:")
for score, chunk_text in retrieve(QUESTION, chunks_with_embeddings, k=1):
```

```
    print(f"  {score:.3f}  {chunk_text!r}")

# Time retrieval at a few values of k.
print("\ntiming at k=1, k=3, k=10:")
for k in (1, 3, 10):
    start = time.perf_counter()
    results = retrieve(QUESTION, chunks_with_embeddings, k)
    elapsed = time.perf_counter() - start
    print(f"k = {k} | time: {elapsed:.4f} seconds")
    for score, chunk_text in results:
        print(f"    {score:.3f}  {chunk_text!r}")
```